

Proyecto PYTHON – FLASK -BD

Panel estadístico educativo con CSV anonimizados

Descripción del proyecto. Se pretende crear una aplicación web con Python y Flask que permita consultar estadísticas educativas útiles para un centro docente a partir de ficheros CSV anonimizados. La aplicación no gestionará datos personales: se eliminarán nombres, apellidos, DNI y cualquier dato identificativo directo. El identificador de trabajo será MATRICULA, que se usará únicamente como código interno no vinculante.

Objetivo final: construir una web con Flask que permita analizar rendimiento por materia, rendimiento por grupo y absentismo mediante tablas, filtros y gráficas.

1. Objetivo general

El alumnado deberá desarrollar una aplicación web en Flask que cargue varios archivos CSV, relacione sus datos mediante pandas y muestre estadísticas educativas de forma clara y útil para la toma de decisiones del centro.

- Trabajar con datos reales anonimizados en formato CSV.
- Relacionar tablas usando columnas comunes.
- Calcular estadísticas con pandas: filtros, agrupaciones, medias, conteos y porcentajes.
- Mostrar resultados en una web con rutas Flask, plantillas Jinja2, tablas HTML y gráficas.
- Organizar el código en funciones reutilizables y mantener una estructura clara del proyecto.

2. CSV que se utilizarán

CSV	Uso en el proyecto	Columnas clave
datNotas.csv	Archivo principal para calcular rendimiento académico.	MATRICULA, ANNO, CURSO, EVALUACION, MATERIA, CL_MATERIA, NOTA
datMatriculas.csv	Permite saber a qué grupo, etapa o estudio pertenece cada matrícula.	MATRICULA, ETAPA, ANNO, ESTUDIOS, GRUPO, ESTADOMATRICULA
datFaltas.csv	Archivo principal para absentismo.	EXPEDIENTE, ANNO, FECHA_FALTA, ESTUDIOS, GRUPO, MATERIA, AUSENCIAS, RETRASOS, JUSTIFICADAS
datMaterias.csv	Permite obtener información descriptiva de las materias.	MATERIA, DESCRIPCION, ABREVIATURA, CURSO
datUnidades.csv	Opcional. Permite ampliar información del grupo/unidad.	ANNO, GRUPO, ESTUDIO, CURSO, TUTOR

3. Cómo relacionar los CSV

La relación entre los ficheros se realizará con pandas mediante `pd.merge()` o cualquier otra técnica que el alumno considere oportuna.

Relación	Para qué sirve
Notas + Matrículas	Añadir grupo, estudios y etapa a cada nota. Es la unión principal del bloque de rendimiento.
Notas + Materias	Añadir descripción/abreviatura oficial de la materia. También se puede usar <code>datNotas.MATERIA</code> si ya viene con texto legible.
Faltas + Matrículas	Relacionar las faltas con la matrícula anonimizada y, si hiciera falta, cruzarlas con otros datos.
Faltas + Materias	Normalizar el nombre de materia en las estadísticas de absentismo. Si no encaja, se puede agrupar directamente por <code>datFaltas.MATERIA</code> .
Matrículas + Unidades	Añadir información de unidad/curso/tutor al análisis por grupo.

3.1. Código orientativo de carga y relaciones

Recomendación para el alumnado: crear un archivo `app/data_loader.py` o `app/services/datos.py` con funciones como `cargar_datos()`, `obtener_rendimiento()` y `obtener_absentismo()`, en lugar de hacer toda la carga dentro de las rutas Flask.

4. Funcionalidades obligatorias

4.1. Funcionalidad 1: rendimiento por materia

Esta sección analizará los resultados académicos agrupando las notas por materia.

Elemento	Descripción
CSV necesarios	<code>datNotas.csv</code> , <code>datMatriculas.csv</code> y <code>datMaterias.csv</code> .
Datos mínimos	Materia, evaluación, curso/estudios, nota y matrícula.
Filtros	Evaluación, curso/estudios y opcionalmente grupo.
Métricas	Número de alumnos evaluados, nota media, aprobados, suspensos, porcentaje de aprobados y porcentaje de suspensos.
Gráficas	Barras de porcentaje de aprobados por materia. Opcional: barras de nota media por materia.
Tabla	Materia, evaluados, media, aprobados, suspensos, % aprobados.

4.2. Funcionalidad 2: rendimiento por grupo

Esta sección permitirá comparar grupos o clases a partir de sus resultados académicos.

Elemento	Descripción
CSV necesarios	<code>datNotas.csv</code> y <code>datMatriculas.csv</code> . Opcionalmente <code>datMaterias.csv</code> .
Relación principal	<code>datNotas.MATRICULA</code> con <code>datMatriculas.MATRICULA</code> .
Filtros	Evaluación, estudios/curso y opcionalmente materia.
Métricas	Número de alumnos evaluados por grupo, nota media del grupo, % aprobados, % suspensos y materias con peor media.
Gráficas	Barras comparativas de nota media por grupo o % aprobados por grupo.
Tabla	Grupo, estudios, evaluados, media, aprobados, suspensos, % aprobados.

4.3. Funcionalidad 3: absentismo

Esta sección analizará ausencias, retrasos y faltas justificadas por grupo, materia y alumno anonimizado.

Elemento	Descripción
CSV necesarios	datFaltas.csv y datMatriculas.csv. Opcionalmente datMaterias.csv.
Relación principal	datFaltas.EXPEDIENTE con datMatriculas.MATRICULA.
Filtros	Rango de fechas, grupo, estudios y materia.
Métricas	Total de ausencias, total de retrasos, justificadas, no justificadas estimadas, faltas por grupo, faltas por materia y faltas por matrícula.
Gráficas	Barras de ausencias por grupo, barras de ausencias por materia y línea temporal por meses.
Tabla	Grupo/materia/matrícula, ausencias, retrasos, justificadas, total de incidencias.

5. Rutas que debe tener la aplicación Flask

La aplicación deberá estar organizada en varias páginas. Las rutas mínimas recomendadas son las siguientes:

Ruta	Método	Página / función	Contenido esperado
/	GET	Dashboard principal	Resumen general: total de matrículas analizadas, número de grupos, número de materias, media global y % global de aprobados.
/materias	GET	Rendimiento por materia	Formulario de filtros y tabla/gráfica de aprobados, suspensos y nota media por materia.
/grupos	GET	Rendimiento por grupo	Formulario de filtros y comparativa de grupos con media y % de aprobados.
/absentismo	GET	Absentismo	Formulario de filtros por fecha, grupo y materia. Tabla/gráficas de ausencias y retrasos.

5.1. Parámetros GET recomendados

Ruta	Parámetros posibles	Ejemplo
/materias	evaluacion, curso, grupo	/materias?evaluacion=1%C2%AA+Evaluaci%C3%B3n&grupo=1%C2%BA+DAW+D
/grupos	evaluacion, estudios, materia	/grupos?evaluacion=2%C2%AA+Evaluaci%C3%B3n&materia=Programaci%C3%B3n
/absentismo	fecha_inicio, fecha_fin, grupo, materia	/absentismo?fecha_inicio=2025-09-01&fecha_fin=2026-01-31&grupo=1%C2%BA+DAW+D

6. Estructura recomendada del proyecto

```
proyecto_flask_estadisticas/
├── app.py
├── requirements.txt
├── data/
│   ├── datNotas.csv
│   ├── datMatriculas.csv
│   ├── datFaltas.csv
│   └── datMaterias.csv
├── static/
│   ├── css/
│   │   └── estilos.css
│   └── js/
│       └── graficas.js (opcional)
└── templates/
    ├── base.html
    ├── index.html
    ├── materias.html
    ├── grupos.html
    ├── absentismo.html
    └── acerca.html
```

7. Requisitos mínimos de la entrega

- La aplicación arranca correctamente con Flask.
- Los CSV se cargan desde la carpeta data/.
- Existe un menú de navegación común en base.html.
- Hay al menos cuatro páginas: dashboard, materias, grupos y absentismo.
- Cada funcionalidad incluye una tabla y al menos una gráfica.
- Se usan filtros mediante formularios GET en al menos dos páginas.
- Se utilizan pandas, merge y groupby de forma correcta.
- El código está separado en funciones, evitando rutas Flask excesivamente largas.
- No se muestran datos personales en la web.

8. Ampliaciones opcionales

- Comparar la evolución entre evaluaciones.
- Detectar materias con mayor dificultad según porcentaje de suspensos.
- Crear un apartado de alumnado en riesgo usando solo MATRICULA anonimizada.
- Exportar tablas filtradas a CSV.
- Añadir login simple para diferenciar usuario administrador y usuario visitante.
- Mejorar el diseño con Bootstrap.
- Generar una base de datos donde se vayan almacenando los datos de resultado según se vayan generando.

9. Criterios de evaluación orientativos

Criterio	Peso orientativo	Qué se valorará
Carga, limpieza y relación de CSV	25%	Uso correcto de pandas, limpieza de columnas, tipos de datos, merge y tratamiento de valores nulos.
Funcionalidades estadísticas	30%	Cálculos correctos en rendimiento por materia, rendimiento por grupo y absentismo.
Aplicación Flask	20%	Rutas, plantillas, formularios, navegación y organización del código.
Visualización de datos	15%	Tablas claras, gráficas comprensibles y filtros útiles.
Presentación y calidad del proyecto	10%	Código limpio, estructura de carpetas, comentarios útiles y ausencia de datos personales.

10. Entrega

- Carpeta completa del proyecto o repositorio comprimido.
- Archivo requirements.txt con las dependencias usadas.
- Breve README.md explicando cómo ejecutar la aplicación.
- Capturas de pantalla de las páginas principales.
- CSV anonimizados usados para las pruebas.